

Lab Assignment 1: Hamming Distance Lab Report

Purpose:

The purpose of this assignment was to write an assembly language program that calculates the hamming distance between two strings to represent the number of bits that are different in both strings.

Approach:

I began this project with a basic conceptual approach by including what variables that will be needed (such as storing the two strings into these variables), converting the strings to its ASCII representations, implementing a loop to iterate through the strings, using XOR to compare the strings to track the bits that were different, storing the hamming distance result into a variable, and printing the output message with the hamming distance result. Then, I began coding for the various sections. For the “.data” section, I included the predefined strings that I wanted to compare, created the output message prompt, and newline character. The “str1” and “str2” will store the two strings that will be compared, “msg” is a string that will be printed with the hamming distance result, and “newline” to store a newline character to print a line break after the result. For the “.bss” section, I reserved 4 bytes of space for the hamming distance result that will be stored in “hamming_distance” and its ASCII representation in “hamming_distance_ascii”. For the “.text” section, it contains the actual code to perform the Hamming distance calculation and output of the result.

The code begins by loading the addresses of the first string and second string into the esi and edi registers to be used as general purpose index registers that point to the addresses of the strings with the ecx register acting as the counter variable that will store the number of bits that are different in both strings. The “compare_strings_loop” retrieves a byte from the first string and second string to load them into the al and ah registers to compare each byte of the strings; the “xor al, ah” operation will compare the two types and sets bits in the al register where the two bytes differ; also, this loop is used to check if the end of each string is reached. After XOR-ing the two bytes, the bl register is used to hold the hamming distance result where it counts how many bits are 1, which indicates a difference. This process as implemented in the “count_num_bits” will shift the bits to the right, add the carry flag into the edx register, and repeat until all the bits in the bl register have been processed. The number of differing bits that is stored in the edx register and is added to the ecx register to accumulate the total hamming distance. The process of moving onto the next byte in both strings and comparing them continues until the end of a string is reached as denoted by the commands “inc esi” and “inc edi”. Once the

comparison is finished, the hamming distance result is then stored in “hamming_distance”. Finally, the hamming distance result between the two inputted strings is displayed onto the screen and then exit the program. Therefore, my code runs without errors, such as segmentation faults; however, the code does not work as intended, in which the hamming distance does not print properly if it is more than one digit, such as in the second test case for the strings “this is a test” and “of the emergency broadcast”.

Code:

```
; Name: Amrita Shrestha
; Lab Assignment 1 - Hamming Distance
; Lab Section: 03
; Date: 3/6/25
```

```
section .data
```

```
; first set of string inputs
```

```
str1 db 'foo', 0 ; First predefined string
```

```
str2 db 'bar', 0 ; Second predefined string
```

```
; second set of string inputs
```

```
;str1 db 'this is a test', 0 ; First predefined string
```

```
;str2 db 'of the emergency broadcast', 0 ; Second predefined string
```

```
; third set of string inputs (custom inputs)
```

```
;str1 db 'dog', 0 ; First predefined string
```

```
;str2 db 'fish', 0 ; Second predefined string
```

```
msg db 'Hamming distance: ', 0 ; Output message
```

```
len equ $ - msg
```

```
newline db 10 ; Newline character
```

```
section .bss
```

```
hamming_distance resd 1 ; Reserves space for Hamming distance result (as a
dword for larger distances)
```

```
hamming_distance_ascii resb 10 ; Reserves space for the ASCII representation of the
Hamming distance
```

```
section .text
```

```
global _start
```

```

_start:
    mov esi, str1                ; Loads the address of first string into the esi register
    mov edi, str2                ; Loads the address of second string into the edi register
    xor ecx, ecx                ; Clears the ecx register to be used as a counter for the
hamming distance

; Compares the two strings
compare_strings_loop:
    mov al, [esi]                ; Loads the next byte of first string into the al register
    mov ah, [edi]                ; Loads the next byte of second string into the ah register
    test al, al                  ; Checks if the end of the string is reached
    jz end_of_calculation        ; If end of the first string is reached (as implied by 0), then
jump to end_of_calculation
    test ah, ah                  ; Checks if end of the second string is reached
    jz end_of_calculation        ; If end of the second string is reached (as implied by 0),
then jump to end_of_calculation
    xor al, ah                    ; Uses XOR bytes to find differing bits

; Counts the differing bits in the al register
    mov bl, al                    ; Copies al register to bl register to count the bits of a byte
    xor edx, edx                  ; Clears the edx register that will be used as a counter for the
bits of this byte

count_num_bits:
    shr bl, 1                    ; Shifts to the right by one bit to move the LSB into the carry
flag
    adc edx, 0                    ; Adds the carry flag to the edx register and counting the bit if
it's set
    test bl, bl                  ; Checks if all bits have been shifted and tested
    jnz count_num_bits          ; If not all bits have been shifted/tested, then continue
counting

    add ecx, edx                  ; Adds the edx and ecx registers to accumulate total Hamming
distance
    inc esi                      ; Moves on to the next character in first string
    inc edi                      ; Moves on to next character in second string
    jmp compare_strings_loop      ; Compares the next set of bits for the first and second
strings

```

```

; After completing the calculation, stores the calculated Hamming distance into
hamming_distance
end_of_calculation:
    mov [hamming_distance], ecx

; Converts Hamming distance to ASCII
movzx eax, byte [hamming_distance]      ; Zero-extend the byte to a dword
add al, '0'                             ; Converts result to ASCII
mov [hamming_distance], al              ; Stores the ASCII character back into ax register

; Outputs the hamming distance onto the terminal
mov eax, 4                             ; system call number (sys_write)
mov ebx, 1                             ; file descriptor (stdout)
mov ecx, msg                           ; message to write
mov edx, len                           ; length of output message
int 0x80                                ; calls kernel

; Outputs the Hamming distance
mov eax, 4                             ; system call number (sys_write)
mov ebx, 1                             ; file descriptor (stdout)
mov ecx, hamming_distance               ; pointer to Hamming distance result
mov edx, 1                             ; length of Hamming distance result
int 0x80                                ; calls kernel

; Outputs the newline
mov eax, 4                             ; system call number (sys_write)
mov ebx, 1                             ; file descriptor (stdout)
mov ecx, newline                       ; pointer to newline character
mov edx, 1                             ; length of newline character
int 0x80                                ; calls kernel

; Exit the program
mov eax, 1                             ; system call to exit
xor ebx, ebx                           ; Exit code 0
int 0x80                                ; calls kernel

```

Outputs:

Note: Each pair of strings were tested separately by commenting/uncommenting the lines corresponding to each test case (containing two strings) in the code

```
[amritas2@linux6 assign1]$ nasm -f elf64 hamming_distance.asm
[amritas2@linux6 assign1]$ ld -o hamming_distance hamming_distance.o
[amritas2@linux6 assign1]$ ./hamming_distance
Hamming distance: 8
```

Hamming Distance Result of the strings “foo” and “bar” (test case 1)

```
[amritas2@linux6 assign1]$ nasm -f elf64 hamming_distance.asm
[amritas2@linux6 assign1]$ ld -o hamming_distance hamming_distance.o
[amritas2@linux6 assign1]$ ./hamming_distance
Hamming distance: V
```

Hamming Distance Result of the strings “this is a test” and “of the emergency broadcast” (test case 2)

```
[amritas2@linux6 assign1]$ nasm -f elf64 hamming_distance.asm
[amritas2@linux6 assign1]$ ld -o hamming_distance hamming_distance.o
[amritas2@linux6 assign1]$ ./hamming_distance
Hamming distance: 5
```

Hamming Distance Result of the strings “dog” and “fish” (custom inputs)

Errors and Attempts to Debug:

One unintended error that I was encountering was when the hamming distance result was more than one digit, it would not print the correct result. For example, the second test case with the inputted strings “this is a test” and “of the emergency broadcast” displayed “V” onto the screen instead of the expected output “38”. I first attempted to hardcode the strings “01110100 01101000 01101001 01110011 00100000 01101001 01110011 00100000 01100001 00100000 01110100 01100101 01110011 01110100” and “01101111 01100110 00100000 01110100 01101000 01100101 00100000 01100101 01101101 01100101 01110010 01100111 01100101 01101110 01100011 01111001 00100000 01100010 01110010 01101111 01100001 01100100 01100011 01100001 01110011 01110100” into its binary equivalents for “this is a test” and “of the emergency” to check if there was a difference in the hamming distance result; however, the same output “V” was displayed into the screen. Also, I thought that adding the ‘0’ to the byte may have caused errors because this may have caused it to incorrectly map to a valid character unless the distance was a single bit. However, I ran out of time to implement other forms of attempts at debugging the code.